ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

A PROJECT REPORT ON

DocuVerify – Cryptographic Certificate Generator & Verification Portal

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Artificial Intelligence & Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Student name	Enrollment number
Devansh Mathur	24EARAD040
Deepak Kumar	24EARAD035
Hansraj Pandey	24EARAD055
Aryan Daware	24EARAD020
Devkinandan Kumawat	24EARAD042

Under the guidance of Er. Ram Babu Buri

Session: 2026-27

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

DOCUMENT CONTROL

Document ID: SC-SRS-001 | Version: 1.0 | Status: Academic Project Draft

Field	Details
Project	SmartCare – Smart Hospital Management System
Prepared by	Team Nobita, Department of AI & Data Science
Reviewed by	Project Guide: Er. Ram Babu Buri
Academic session	2026–27
Technology baseline	Java 17, Jakarta Servlets/JSP, JDBC, MySQL, Maven WAR; BCrypt; Docker deployment if used

CONTENTS

1. Introduction and Scope
2. Overall Description
3. System Architecture
4. Functional Requirements
5. Non-Functional Requirements
6. Data and External Interface Requirements
7. Data Flow Diagrams (DFD)
8. Entity Relationship Diagram (ERD)
9. Use-Case Model and Workflows
10. Verification, Acceptance and Traceability
11. Assumptions, Constraints and Risks
12. Glossary and Conclusion

Document note: This SRS has been aligned to the project files supplied for SmartCare. Diagrams are logical representations of the described application and database schema. Requirements and test criteria must be verified against the final deployed build before being claimed as passed.

1. INTRODUCTION AND SCOPE

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for SmartCare, a web-based hospital management application. It establishes a common reference for the development team, guide, testers and academic evaluators, and provides a basis for deriving test cases and reviewing system completeness.

1.2 Product Scope

The supplied project structure indicates modules for login and role management, patient and doctor management, appointment scheduling, medical records and consultation, prescriptions, medicines and inventory, laboratory tests, nursing and patient vitals, admissions and rooms, billing and payments, feedback, audit logs and notifications. These modules are presented as the project scope because corresponding database entities and application components are present in the supplied project archive. Feature details must still be confirmed in the running build.

1.3 Intended Audience

This document is intended for developers, testers, project guides and academic evaluators. Developers use it to understand requirements; testers use it to design verification scenarios; evaluators use it to assess whether the documented scope is supported by the implementation and test evidence.

1.4 Definitions and Requirement Notation

FR-x denotes a functional requirement; NFR-x denotes a non-functional requirement; PK denotes primary key; FK denotes foreign key; DFD denotes Data Flow Diagram; ERD denotes Entity Relationship Diagram. “Shall” indicates a required behaviour in the defined scope. “Should” indicates a recommended quality or implementation practice.

1.5 System Boundary

SmartCare is an academic software project for hospital administration and record workflows. This SRS does not claim clinical validation, regulatory certification, integration with external national health platforms, or suitability for real-world clinical decision-making.

2. OVERALL DESCRIPTION

2.1 Product Perspective

The project follows a Java web-application structure. Browser requests are handled by JSP-based presentation pages and servlet controllers. Authentication and authorization filters protect routes, DAO components communicate with the MySQL database through JDBC, and the database schema defines the principal entities and relationships. Maven is used for build packaging; Docker may be used where included in the deployment setup.

2.2 User Classes

Administrator: manages system-level users, roles and administrative data.

Doctor: accesses appointment, patient, consultation, medical-record, prescription and laboratory information permitted by the application.

Nurse: accesses permitted patient and vital-record workflows.

Receptionist: manages patient registration, appointments, rooms/admissions and billing workflows supported by the application.

Patient: accesses permitted profile, appointments, medical records, prescriptions, laboratory reports, bills and feedback pages.

2.3 Operating Environment

The application is intended to be accessed through a modern web browser. The project stack specifies Java 17, Jakarta Servlets/JSP, JDBC and MySQL. Exact servlet-container, browser and server versions should be recorded from the final deployment environment.

2.4 Constraints and Dependencies

The application depends on a running Java web container, valid configuration, reachable MySQL database and a correctly created schema. Some features depend on linked records—for example, appointments depend on patients and doctors, prescriptions depend on patients and doctors, and payments depend on bills. The actual role permissions must match the supplied authorization logic.

2.5 Assumptions

Users have valid accounts and use only their assigned roles. Test data is synthetic. The final implementation may differ from the intended scope, so requirements must be reconciled against the running system and source code before formal acceptance.

3. SYSTEM ARCHITECTURE

Figure 1 presents a logical view of the application's main layers. It is based on the supplied Java web application structure and is intended to show responsibility boundaries rather than every class or deployment component.

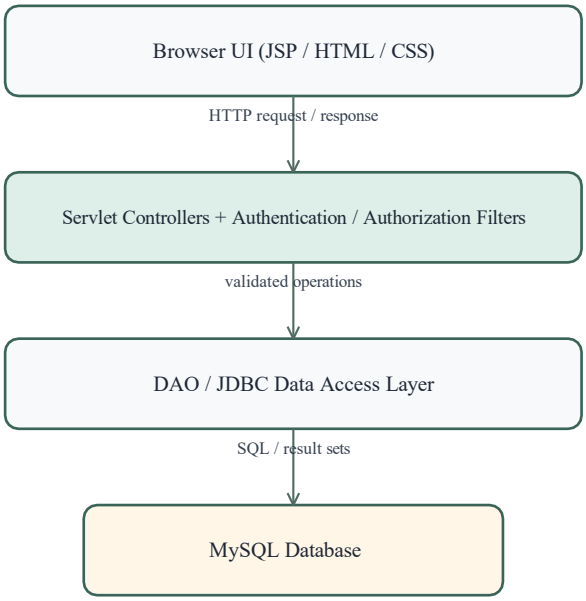


Figure 1. Logical SmartCare application architecture.

3.1 Presentation Layer

JSP pages and browser-facing components display forms, dashboards, lists and status messages. The interface should use consistent labels, predictable navigation and clear validation feedback.

3.2 Controller and Access-Control Layer

Servlet controllers process incoming requests and invoke the appropriate application/data operations. Authentication filters verify session state; authorization logic restricts access based on role and route/action permissions. Server-side checks are required even when the interface hides unavailable actions.

3.3 Data-Access Layer

DAO classes encapsulate database operations and map stored records to application models. Database operations should use safe parameter handling, appropriate transaction boundaries and predictable error handling.

3.4 Persistence Layer

MySQL stores users, roles, patients, doctors, appointments, medical records, medicines, prescriptions, lab tests, admissions, billing and other supported records. Referential integrity is expressed through primary keys and foreign keys in the schema.

4. FUNCTIONAL REQUIREMENTS

4.1 Authentication and Roles

FR-01: The system shall allow registered users to authenticate using valid credentials.

FR-02: Invalid credentials shall not grant access and shall produce a non-sensitive message. FR-03: An authenticated user shall be able to log out.

FR-04: Protected routes shall require an authenticated session.

FR-05: Access to functions shall be restricted according to the user's assigned role.

FR-06: Passwords shall not be stored or displayed in plain text; the supplied project describes BCrypt-based password handling.

4.2 Patient and Doctor Management

FR-07: Authorised users shall be able to create, view, search and update patient records within the implemented fields.

FR-08: Authorised users shall be able to view and maintain doctor information and department associations supported by the application.

FR-09: The system shall validate required fields and reject invalid values.

FR-10: The system should prevent accidental duplicate records when a reliable unique identifier is available.

4.3 Appointment Management

FR-11: Authorised users shall be able to create appointments associated with a patient and doctor. FR-12: Appointment data shall be validated before saving.

FR-13: The system shall support viewing appointment details and status for authorised roles.

FR-14: Rescheduling, cancellation and conflict prevention shall follow the rules implemented in the application and must be tested explicitly.

4.4 Medical Records and Prescriptions

FR-15: Authorised clinicians shall be able to create or view medical records associated with the relevant patient and doctor. FR-16: Medical records may be linked to an appointment where the schema supports that relationship.

FR-17: Authorised users shall be able to create and view prescriptions for the relevant patient and doctor. FR-18: Prescription items shall reference medicines supported by the database schema.

4.5 Laboratory and Pharmacy

FR-19: Authorised users shall be able to manage laboratory test records and status/results supported by the application. FR-20: Lab records shall be associated with the relevant patient and doctor and optionally appointment where supported. FR-21: Authorised users shall be able to view medicine information and inventory records.

FR-22: If inventory quantity changes are implemented, values shall be validated and stored consistently.

4.6 Nursing, Admissions and Rooms

FR-23: Authorised nursing users shall be able to record and view patient vital information supported by the application. FR-24: Vital records shall reference the relevant patient and nurse.

FR-25: Authorised staff shall be able to manage admissions and room associations where implemented. FR-26: Admission records shall reference the appropriate patient and, where applicable, doctor and room.

4.7 Billing and Payments

FR-27: Authorised staff shall be able to create and view billing records supported by the application.

FR-28: Billing records shall be linked to the relevant patient and may reference an appointment or admission where applicable. FR-29: Payment records shall reference the bill to which the payment belongs.

FR-30: Monetary fields and calculated totals shall be validated and tested against known examples.

4.8 Feedback, Audit Logs and Notifications

FR-31: The system shall support patient feedback records as implemented, with optional association to a doctor where supported.

FR-32: Audit events should record relevant user actions where the audit module is implemented.

FR-33: Notifications shall be associated with the intended user and displayed only to authorised recipients. FR-34: Successful and failed actions should return clear status messages.

4.9 Reports and Search

FR-35: The system shall expose only the reports and search features actually provided by the application. FR-36: Search and report results shall respect user permissions and use consistent filtering rules.

FR-37: Empty result sets and invalid filters shall be handled with clear messages rather than misleading output.

4.10 Error Handling

FR-38: The application shall handle invalid input and unexpected server/database errors without displaying sensitive implementation details.

FR-39: A failed database operation shall not be presented to the user as a successful save.

FR-40: Access-denied, not-found and internal-error pages should provide an appropriate response without exposing secrets.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Security

NFR-01: Authentication shall be required for protected functions. NFR-02: Authorization shall be enforced on the server side.

NFR-03: Passwords shall be protected using the configured password-hashing method. NFR-04: Database queries should use parameterized statements where applicable.

NFR-05: Credentials and secret configuration values shall not be committed to public source code. NFR-06: Logs and user-facing errors shall avoid unnecessary sensitive information.

NFR-07: Sessions shall be handled safely, including logout and expired-session behaviour.

5.2 Usability and Accessibility

NFR-08: Pages shall use consistent navigation, labels and terminology. NFR-09: Form errors shall identify the invalid or missing input clearly. NFR-10: Save/update actions should provide visible confirmation.

NFR-11: Text, form controls and status indicators should remain readable and usable across common screen sizes.

5.3 Performance

NFR-12: Common actions should meet response-time targets defined before performance testing.

NFR-13: Performance reports shall document environment, dataset size, number of repetitions and observed response times. This SRS does not invent a numerical target or claim measured performance.

5.4 Reliability and Data Integrity

NFR-14: The system should handle database unavailability and invalid input without silent data loss. NFR-15: Related records shall maintain the primary-key/foreign-key relationships defined in the schema. NFR-16: Critical data changes should be traceable where audit logging is implemented.

5.5 Maintainability and Portability

NFR-17: Source code should maintain clear separation between presentation, controller and data-access responsibilities. NFR-18: Build and setup instructions should be documented and repeatable.

NFR-19: Schema changes should be versioned or documented.

NFR-20: Deployment-specific configuration should be documented separately from source code.

5.6 Privacy and Ethical Use

NFR-21: Demonstrations and testing should use synthetic data unless authorised data use has been approved. NFR-22: Patient-related information shall be accessible only for authorised purposes.

NFR-23: The academic prototype shall not be used as a substitute for professional medical systems or clinical judgement.

6. DATA AND EXTERNAL INTERFACE REQUIREMENTS

6.1 User Interface Requirements

The interface shall provide sign-in, role-specific navigation and the pages supported by the application. Forms should label required fields, validate user input and preserve understandable state after errors. Dangerous or irreversible actions should be confirmed where applicable. Patient identifiers and appointment details should remain clear throughout multi-step workflows.

6.2 Software Interfaces

The supplied project uses Java 17, Jakarta Servlets/JSP, JDBC and MySQL, with Maven WAR packaging. BCrypt is described for password hashing. Docker is included as a deployment-related technology in the project context; whether it is used in the final deployment should be confirmed from the deployment files.

6.3 Database Entities

The supplied schema includes roles, users, departments, doctors, patients, nurses, receptionists, rooms, admissions, appointments, medical_records, medicines, medicine_inventory, prescriptions, prescription_items, lab_tests, billing, payments, patient_vitals, feedback, audit_logs and notifications. Their key relationships are summarised in the ER diagram in Section 8.

6.4 Communication and Deployment

The browser sends requests to the Java web application, which accesses MySQL through JDBC. Deployment settings must define the correct database endpoint and credentials without exposing secrets. Any third-party service should be included in this section only if it is actually configured and used.

6.5 Backup and Recovery

For reproducible demonstrations, the team should document how to create and restore a database backup and how to reset synthetic test data. Recovery steps and responsible access should be documented for the environment used in the evaluation.

7. DATA FLOW DIAGRAMS (DFD)

7.1 DFD Level 0 — Context Diagram

The context diagram treats SmartCare as one process and shows the main external actors exchanging information with it. Arrows indicate representative data flows, not a complete API specification.

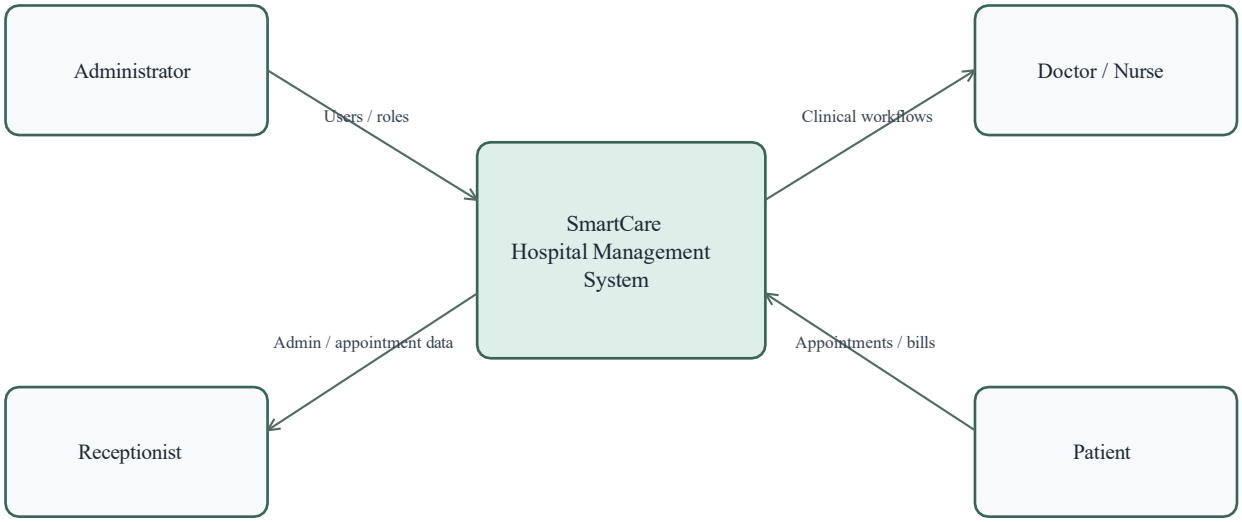


Figure 2. DFD Level 0 / system context diagram.

External entities represented: administrator, receptionist, doctor/nurse and patient. Example flows include credentials and administration requests, patient and appointment data, clinical workflow information, and patient-facing appointment or billing information.

7.2 DFD Level 1 — Major Processes and Data Stores

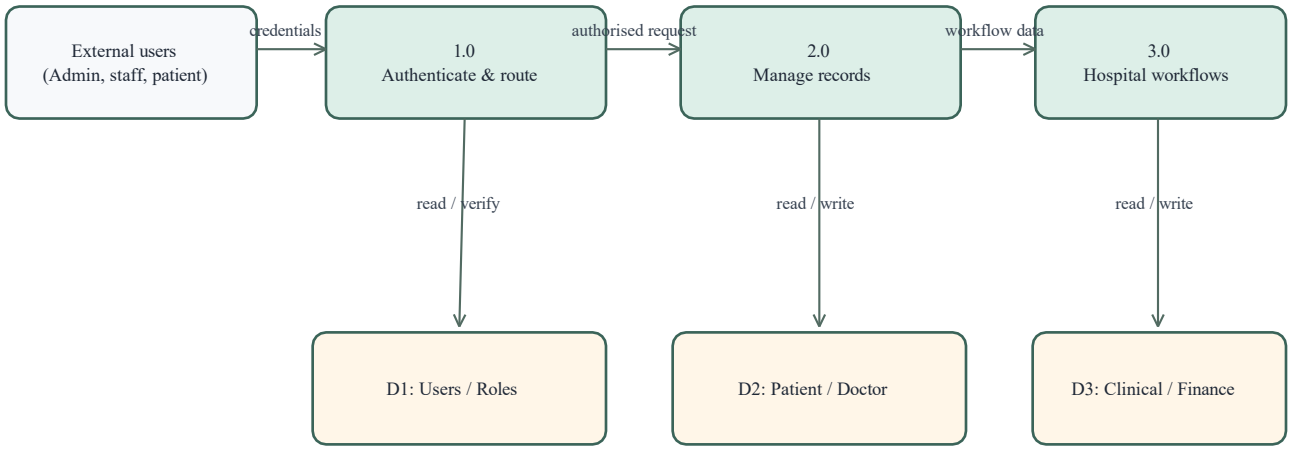


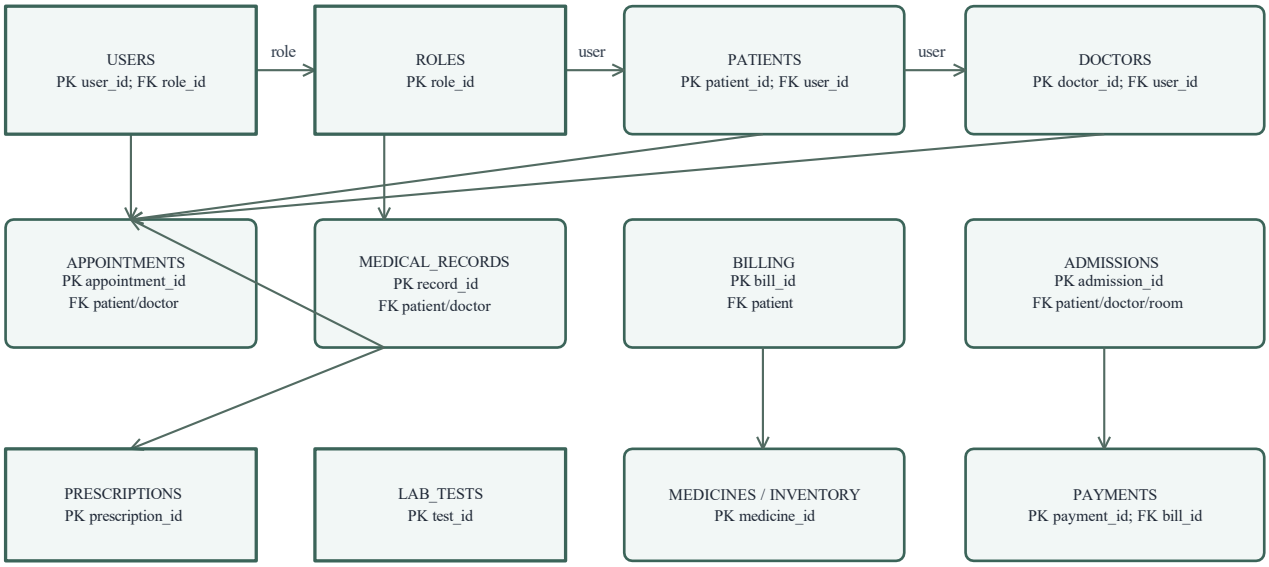
Figure 3. DFD Level 1, simplified logical data flow.

Process 1.0 authenticates the user and routes permitted requests. Process 2.0 manages patient and doctor records. Process 3.0 handles operational workflows such as appointments, clinical records, admissions, pharmacy, laboratory and billing. D1, D2 and D3 are logical groupings of persistent data, not literal database table names.

DFD note: the diagrams are intentionally high-level. They should be updated if the final project introduces additional actors, external services or module boundaries.

8. ENTITY RELATIONSHIP DIAGRAM (ERD)

The logical ERD below summarises principal entities and selected foreign-key relationships from the supplied schema.sql. To keep the figure readable on a portrait page, it focuses on the main patient-care and billing chain; the complete schema contains additional supporting entities.



Simplified logical ERD: major entities only; full schema also includes nurses, receptionists, departments, rooms, prescription_items, patient_vitals, feedback, audit_logs and notifications.

Figure 4. Simplified logical ER diagram based on the supplied database schema.

8.1 Principal Relationships

- roles 1-to-many users: each user references a role.
- users 1-to-0/1 profile records: doctors, patients, nurses and receptionists reference users.
- departments 1-to-many doctors and, where assigned, nurses.
- patients and doctors are referenced by appointments, medical records, prescriptions and laboratory records.
- appointments may be referenced by medical records, lab tests and billing records.
- prescriptions 1-to-many prescription_items; each item references a medicine.
- patients may have admissions, bills, vitals, feedback and other records.
- billing 1-to-many payments; admissions may reference rooms.

8.2 Data Integrity Rules

Primary keys uniquely identify records. Foreign keys should prevent references to nonexistent parent records, subject to the cascade and nullification rules defined in schema.sql. Deletion behaviour should be tested carefully because some relationships cascade deletion while others set an optional foreign key to NULL. The ERD is a logical overview and does not replace the SQL schema.

9. USE-CASE MODEL AND CORE WORKFLOWS

9.1 Use-Case Diagram

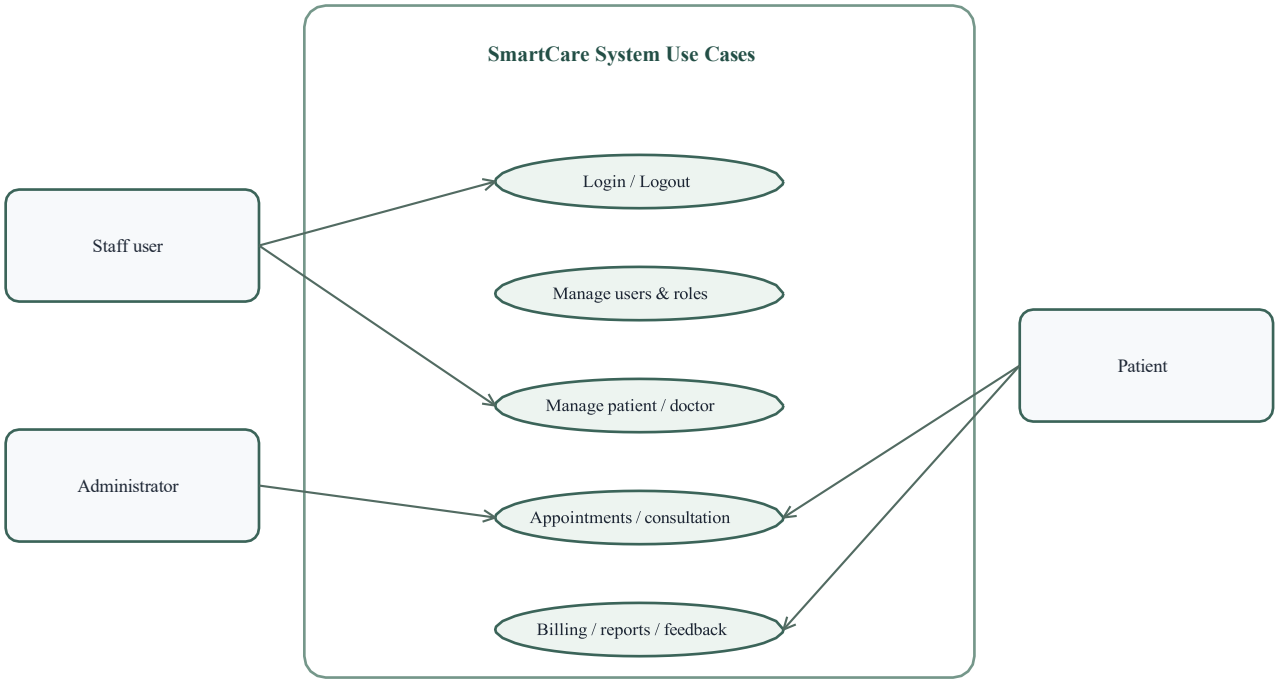


Figure 5. High-level use-case view of SmartCare.

The use-case diagram summarises major interactions. Detailed permissions must be taken from the application's authorization filter and servlet/controller logic. It should not be assumed that every actor can execute every use case.

9.2 Use Case: Sign In

Actor: Any registered user. **Precondition:** Application and database are available.

Main flow: User opens login page, enters credentials, submits; system verifies credentials and establishes a session; the appropriate permitted landing page is shown.

Alternate flow: Invalid credentials result in a generic error and no authenticated session.

Postcondition: Access is granted only for a valid user and assigned role.

9.3 Use Case: Book Appointment

Actor: Patient or authorised receptionist, according to the implemented workflow. **Precondition:** Patient and doctor records exist.

Main flow: Actor selects patient/doctor and enters appointment details; system validates input; appointment is saved and displayed.

Alternate flow: Missing fields, invalid references or unsupported time values are rejected.

Postcondition: A saved appointment references the intended patient and doctor.

9.4 Use Case: Clinical Record and Prescription

Actor: Authorised doctor. **Precondition:** The relevant patient and appointment/encounter exist as required.

Main flow: Doctor opens the patient/consultation workflow, records supported medical information and creates a prescription if needed; the system validates and stores records with the correct references.

Postcondition: Permitted users can retrieve the resulting records; unauthorised users cannot access restricted details.

10. VERIFICATION, ACCEPTANCE AND TRACEABILITY

10.1 Test Levels

Unit testing verifies individual components where tests exist. Integration testing checks servlet/controller, DAO and database interactions. System testing exercises complete browser workflows. Security testing checks authentication, authorization, session behaviour and input handling. Usability testing observes representative users completing tasks. Regression testing reruns impacted scenarios after changes.

10.2 Suggested Acceptance Criteria

The project should be considered ready for academic demonstration only after the agreed core workflows have been executed and documented. At minimum, test valid/invalid login, logout, role restrictions, patient record creation/retrieval, appointment creation and relationships, medical record access, billing calculations where implemented, form validation, error handling and build/deployment steps. Criteria should be marked Pass only when supported by recorded evidence.

Requirement area	IDs	Evidence to collect
Authentication & roles	FR-01–06; NFR-01–07	Valid/invalid login, logout, restricted-route tests
Patient / doctor	FR-07–10	Create, search, update, validation results
Appointments	FR-11–14	Appointment creation, linkage and invalid-input cases
Clinical / pharmacy / lab	FR-15–22	Record relationships and permitted-role access
Nursing / admissions	FR-23–26	Vital record and admission/room relationship tests
Billing / payments	FR-27–30	Totals, payment-to-bill link and invalid values
Feedback / audit / notifications	FR-31–34	Submission, user linkage, logs and permissions

10.3 Test Case Template

Test ID | Requirement ID | Preconditions | Test data | Steps | Expected result | Actual result | Pass/Fail | Evidence | Tester/date. Record actual results after execution; do not populate the result column with assumed outcomes.

10.4 Performance and Security Evidence

For performance, record test environment, dataset size, repetitions and observed response times. For security, record role/access checks, session tests and input-validation results. A student checklist does not constitute a professional penetration test or compliance certification.

11. ASSUMPTIONS, CONSTRAINTS AND RISKS

11.1 Key Risks

Incorrect role checks may expose functions or records to unauthorised users. Inconsistent foreign-key relationships may associate data with the wrong patient. Database or deployment configuration errors may prevent the application from starting. Incomplete validation may allow invalid values. Insufficient testing may leave workflow defects undiscovered. These are potential risks to be assessed, not claims that such defects are present.

11.2 Mitigations

Test role permissions on the server side; validate references and input; exercise complete multi-module workflows; use synthetic test data; document configuration and build steps; test backup/restore procedures; review logs for sensitive data; and maintain traceability between requirements and test cases.

11.3 Out of Scope

Unless explicitly implemented and independently validated, the following are out of scope: clinical decision support, automated diagnosis, medical-device integration, insurance claim processing, national health-platform integration, formal regulatory certification and production clinical deployment.

12. GLOSSARY AND CONCLUSION

12.1 Glossary

Authentication: verifying identity. **Authorization:** deciding permitted actions. **BCrypt:** password-hashing algorithm. **DAO:** Data Access Object. **DFD:** Data Flow Diagram. **ERD:** Entity Relationship Diagram. **FK:** Foreign Key. **JDBC:** Java Database Connectivity. **PK:** Primary Key. **RBAC:** Role-Based Access Control. **SRS:** Software Requirements Specification.

12.2 Conclusion

This SRS defines a structured requirements baseline for SmartCare based on the supplied project context and database schema. It includes the project scope, user roles, system architecture, functional and non-functional requirements, DFDs, a logical ERD, use cases, and a test-traceability approach. The diagrams are intended to communicate system structure clearly; the SQL schema and source code remain authoritative for exact implementation details.

Before submission, the team should verify the current build, confirm implemented features and role permissions, record actual test evidence, and update this document to reflect any differences. No unexecuted test is represented as passed.

Prepared by: Team Nobita | **Guide:** Er. Ram Babu Buri | **Session:** 2026–27